Git & Python environments

•••

01ADS

From commits and pull requests to reproducible Python projects with uv.

Quiz

How many of you

- made a commit?
- pushed code?
- used a branch?
- merged a pull request?
- reviewed AI-generated code?
- resolved a merge conflict?

Definition

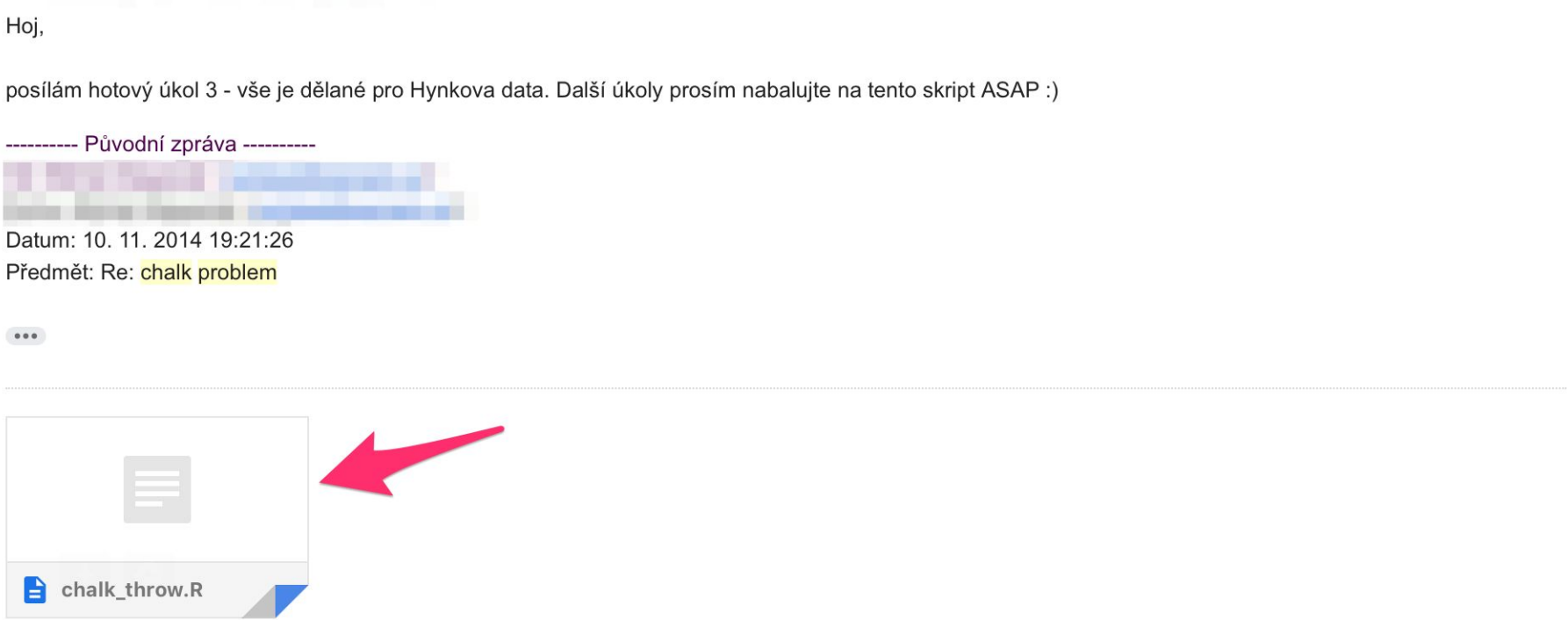
Version control is the practice of tracking and managing changes to software code.

The tool we use for that purpose - Version Control System (VCS).

Motivation

A complete, long-term change history of every file - stored in one place

- Return to previous versions and investigate the root cause of bugs.



Traceability

- Be able to trace each change made to the software and be able to annotate each change with a message describing the purpose and intent - possible to onboard new developers

Nazdar,

Dorobili sme s Viktorom ulohu c. 2. a updatli sme skirpit aby tam bol aj Hynkov aj Michalov kod, tak prosim pouzivajte dalej tento posledny subor.

Odhadovali sme main effect a interaction effects hodnoty, nieco vychadzalo z linearneho modelu, ale realna velkost efektov je 2x vacsia. Je to okomentovane Viktorom, ak sa vam bude chciet tak si to prejdite :)

Hynku, ty ak chces, tak si pozri ten lm co sme tam robili a mas dalsiu vec co mozes porovnat all data vs. single data - je to na dva riadky a tiez celkom lahke okomentovat :)

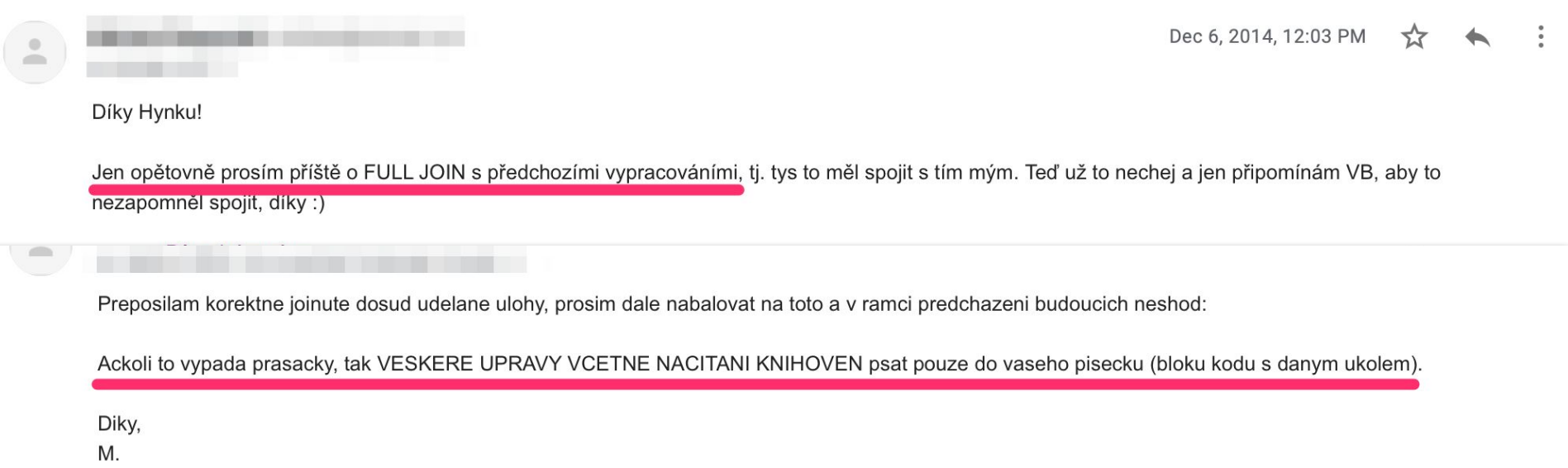
[redacted]

•••



Branching and merging

- Team members can work concurrently, even individuals working on their own can work on independent streams of changes.
- Branching keeps multiple streams of work independent from each other while also providing the facility to merge that work back together, enabling developers to verify that the changes on each branch do not conflict.



[redacted] Dec 6, 2014, 12:03 PM

Díky Hynku!

Jen opětovně prosím příště o FULL JOIN s předchozími vypracováními, tj. tys to měl spojit s tím mým. Teď už to nechej a jen připomínám VB, aby to nezapomněl spojit, díky :)

[redacted]

Preposilam korektne joinute dosud udelane ulohy, prosim dale nabalovat na toto a v ramci predchazeni budoucich neshod:

Ackoli to vypada prasacky, tak VESKERE UPRAVY VCETNE NACITANI KNIHOVEN psat pouze do vaseho pisecku (bloku kodu s danym ukolem).

Diky,
M.

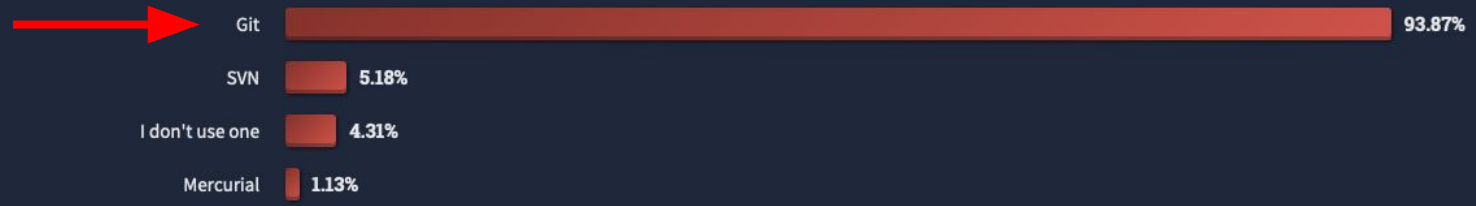

git

Origins

- Author: Linus Torvalds (Linux kernel)
- Meaning of the name:
  - an unpleasant, silly, incompetent, annoying, senile, elderly or childish person.
- Created to manage the Linux kernel source code.

All Respondents | Professional Developers | Learning to Code

71,379 responses



What are the primary **version control systems** you use? Select all that apply.

% Percentages # Responses

All Respondents | Professional Developers | Learning to Code

53,374 responses



What are the primary **version control systems** you use? Select all that apply.

% Percentages # Responses

Interacting with version control systems

The command line is the primary way developers interact with their version control system

All Respondents | Professional Developers | Learning to Code

68,156 responses



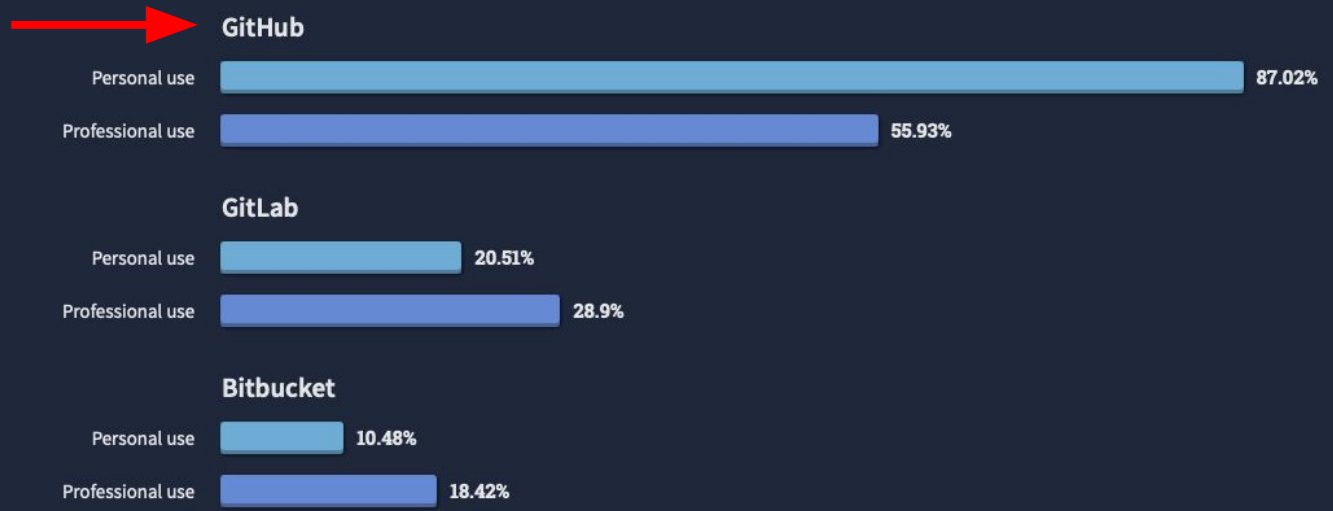
How do you interact with your version control system? Select all that apply.

% Percentages | # Responses

Version control platforms

GitHub is the most popular Version Control for both personal and professional use. GitLab, Bitbucket, and Azure Repos are more likely used for professional purposes instead of personal.

67,035 responses



Git in 2026: AI accelerates coding, not accountability

BEFORE AI

The bottleneck was writing code and searching documentation.

WITH AI

The bottleneck is now context, review, testing and integration.

Generated code enters Git like any other change: small diff, tests, review, evidence.

GitHub Docs: responsible use

GIT LEVEL 1



Basic Terminology (local)

Repository (aka "repo")

- represents project that will be versioned
- is a branching history of snapshots

Commit

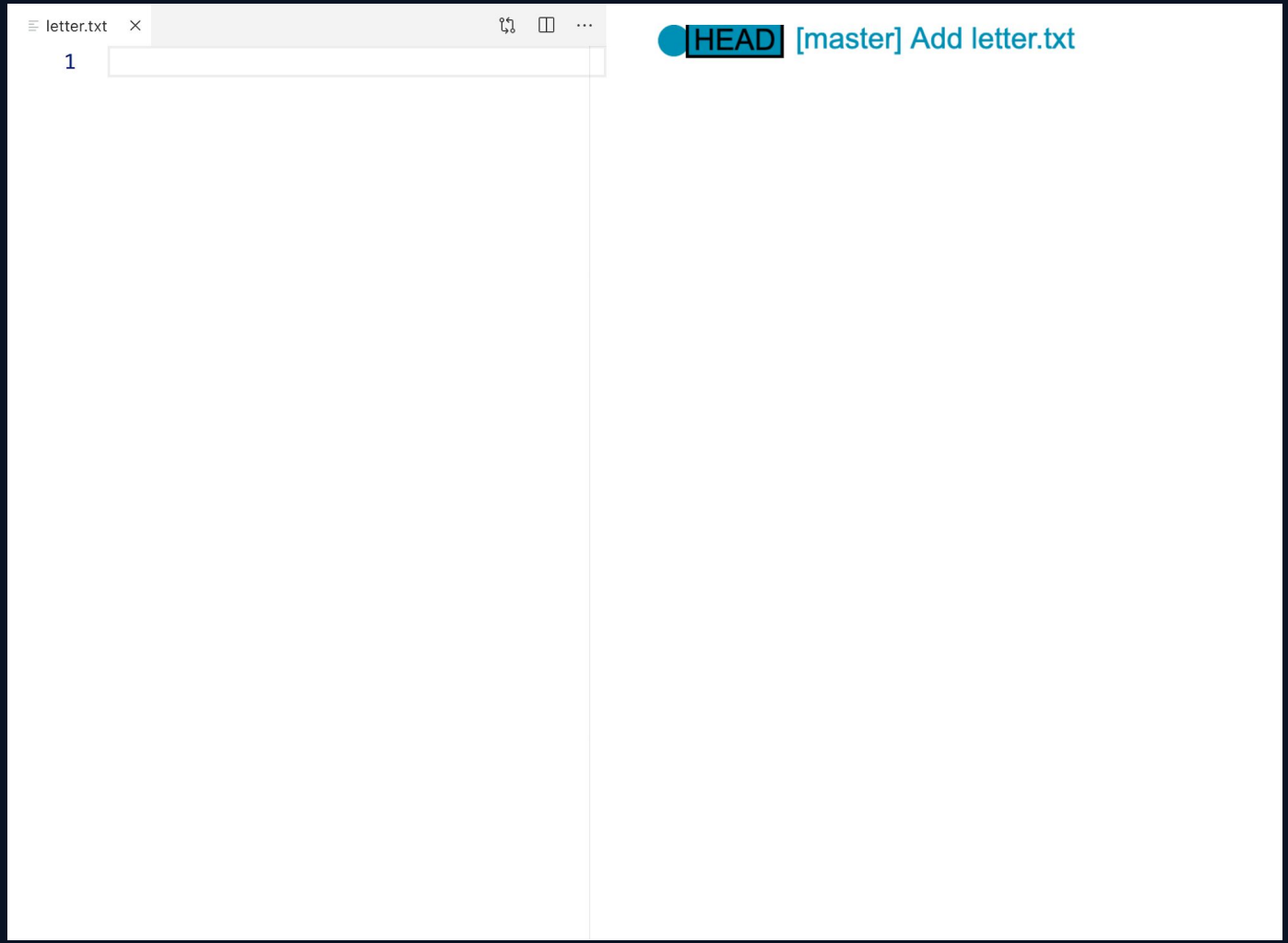
- snapshot of state of the repo (not a diff! -- more on this later)
- should contain atomic change

Branch

- Alternative version of the repository
- “branch fast and often”

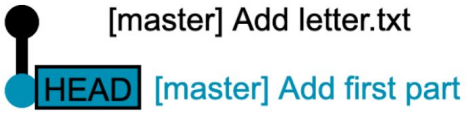
HEAD

- Currently active snapshot


letter.txt
1
HEAD [master] Add letter.txt

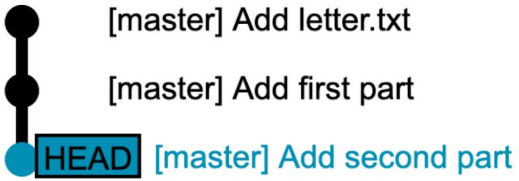
letter.txt

```
1  Dear Mr. Smith:
2
3  Let me begin by thanking you for your
   past contributions.
4
```



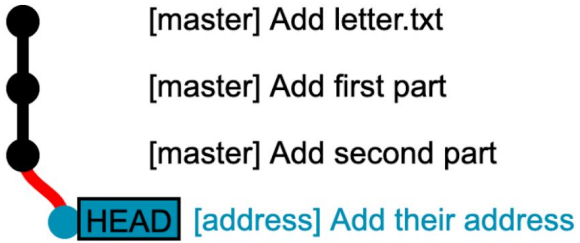
letter.txt

```
1  Dear Mr. Smith:
2
3  Let me begin by thanking you for your
   past contributions.
4  I would like to place an order with
   your company for 25 pounds of pancake.
5
```



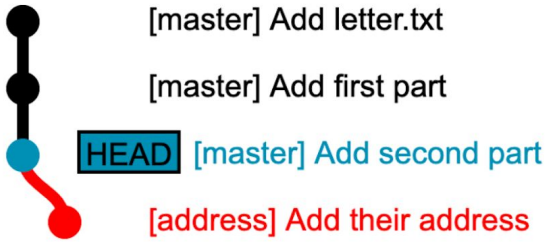
letter.txt

```
1  Mr. A. Smith
2  Chief of Syrup Production
3  Old Sticky Pancake Company
4  456 Maple Lane
5  Forest, ON 7W8 9Y0
6
7
8  Dear Mr. Smith:
9
10 Let me begin by thanking you for your
   past contributions.
11 I would like to place an order with
   your company for 25 pounds of pancake.
12
```



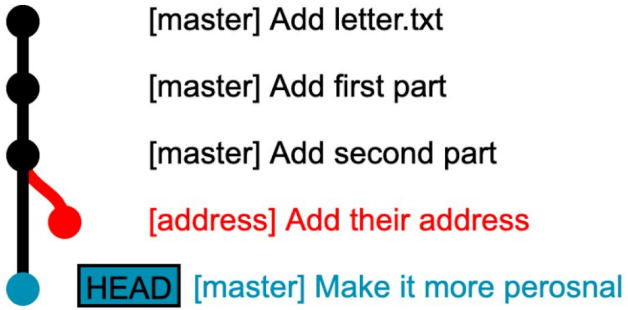
letter.txt

```
1  Dear Mr. Smith:
2
3  Let me begin by thanking you for your
   past contributions.
4  I would like to place an order with
   your company for 25 pounds of pancake.
5
```



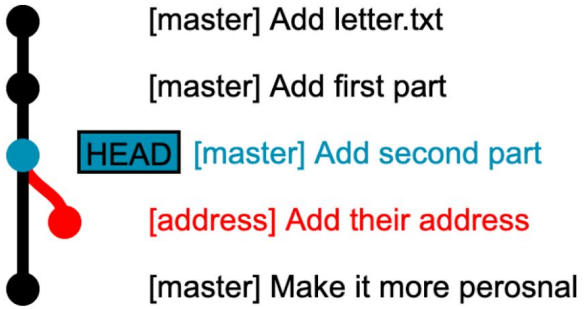
letter.txt

```
1  Hey Bro:
2
3  Let me begin by high fives all around.
4  Gimme pls 25 pounds of pancake.
5
```



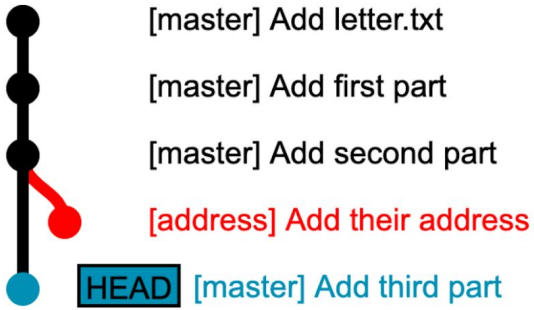
letter.txt

```
1  Dear Mr. Smith:
2
3  Let me begin by thanking you for your
   past contributions.
4  I would like to place an order with
   your company for 25 pounds of pancake.
5
```



letter.txt

```
1  Dear Mr. Smith:
2
3  Let me begin by thanking you for your
   past contributions.
4  I would like to place an order with
   your company for 25 pounds of pancake.
5  I hope that you will be able to join
   us for breakfast next Friday.
6
```



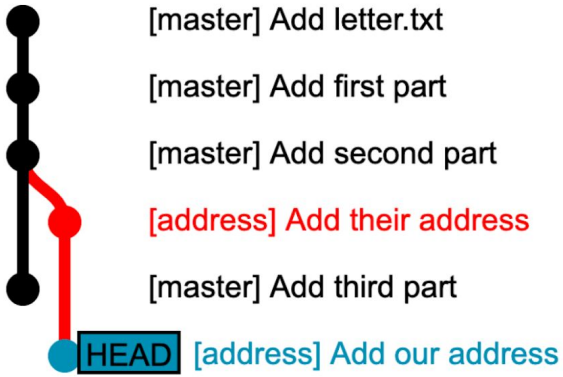
letter.txt

```
123 Elm Ave.
Treesville, ON M1N 2P3
November 7, 2016

Mr. A. Smith
Chief of Syrup Production
Old Sticky Pancake Company
456 Maple Lane
Forest, ON 7W8 9Y0


Dear Mr. Smith:

Let me begin by thanking you for your past contributions.
I would like to place an order with your company for 25 pounds of pancake.
```



letter.txt

```
123 Elm Ave.
Treesville, ON M1N 2P3
November 7, 2016

Mr. A. Smith
Chief of Syrup Production
Old Sticky Pancake Company
456 Maple Lane
Forest, ON 7W8 9Y0


Dear Mr. Smith:

Let me begin by thanking you for your past contributions.
I would like to place an order with your company for 25 pounds of pancake.
I hope that you will be able to join us for breakfast next Friday.
```



[master] Add letter.txt

[master] Add first part

[master] Add second part

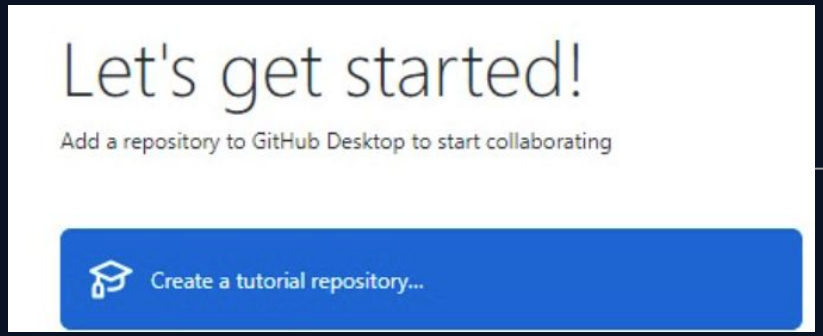
[address] Add their address

[master] Add third part

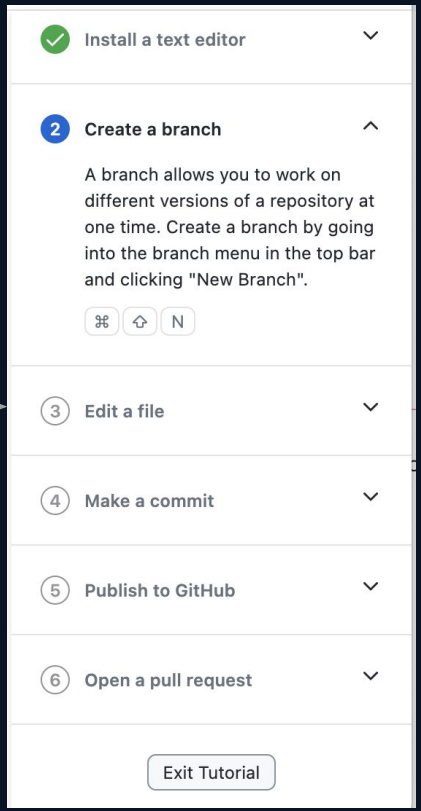
[address] Add our address

[master] Merge to master

Exercise 0: GitHub Desktop tutorial



[Win OS guide]



Exercise 0: “Uncertain poet”

Help the poet add back verses he removed in previous drafts of the poem.

1. Fire up GitHub Desktop (or whatever you use)
    a. If you’re new, do the tutorial
2. Check out https://GitHub.com/Joelino/git-exercise1
3. Find point in history just before the poet removed few verses.
4. Create branch on that commit.
5. Explore the repository and copy the removed verses to the clipboard.
6. Checkout back main branch, paste the missing verses in correct place and commit.

Exercise 1 continued: How could we work more effectively?

1. We did not need a new branch just to inspect history.
    a. Checkout commit “git checkout COMMIT_SHA_HASH”

2. We could restore the removed content from history or revert the removal commit.
    a. NOTE: this would introduce conflict - we will talk about it later.

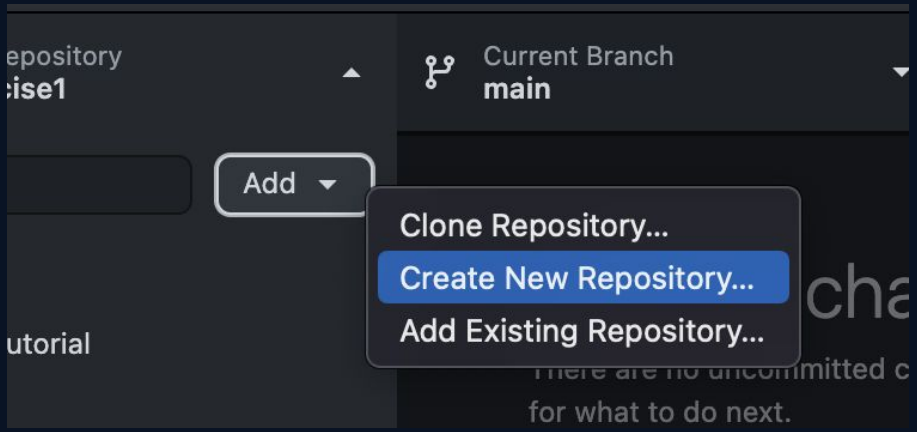
Set up a repository

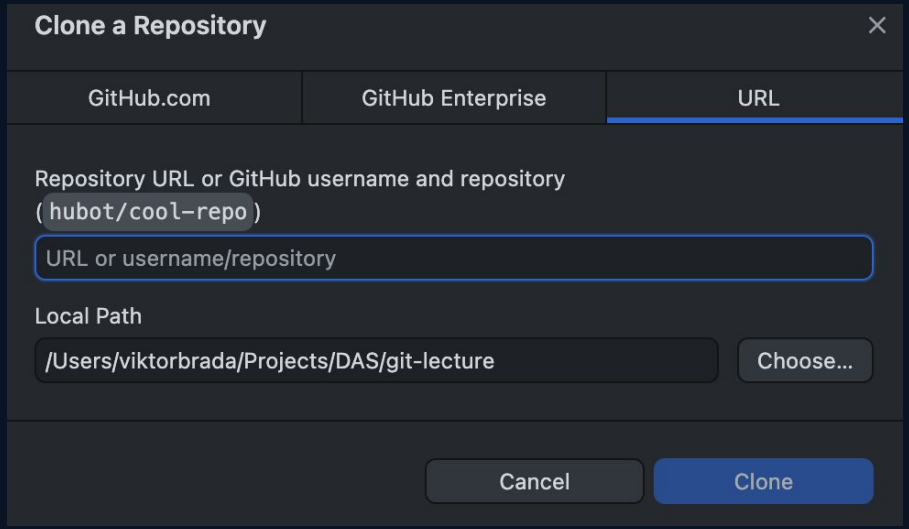
init

- create new repository (or do it in GitHub UI)

clone

- download the full repo



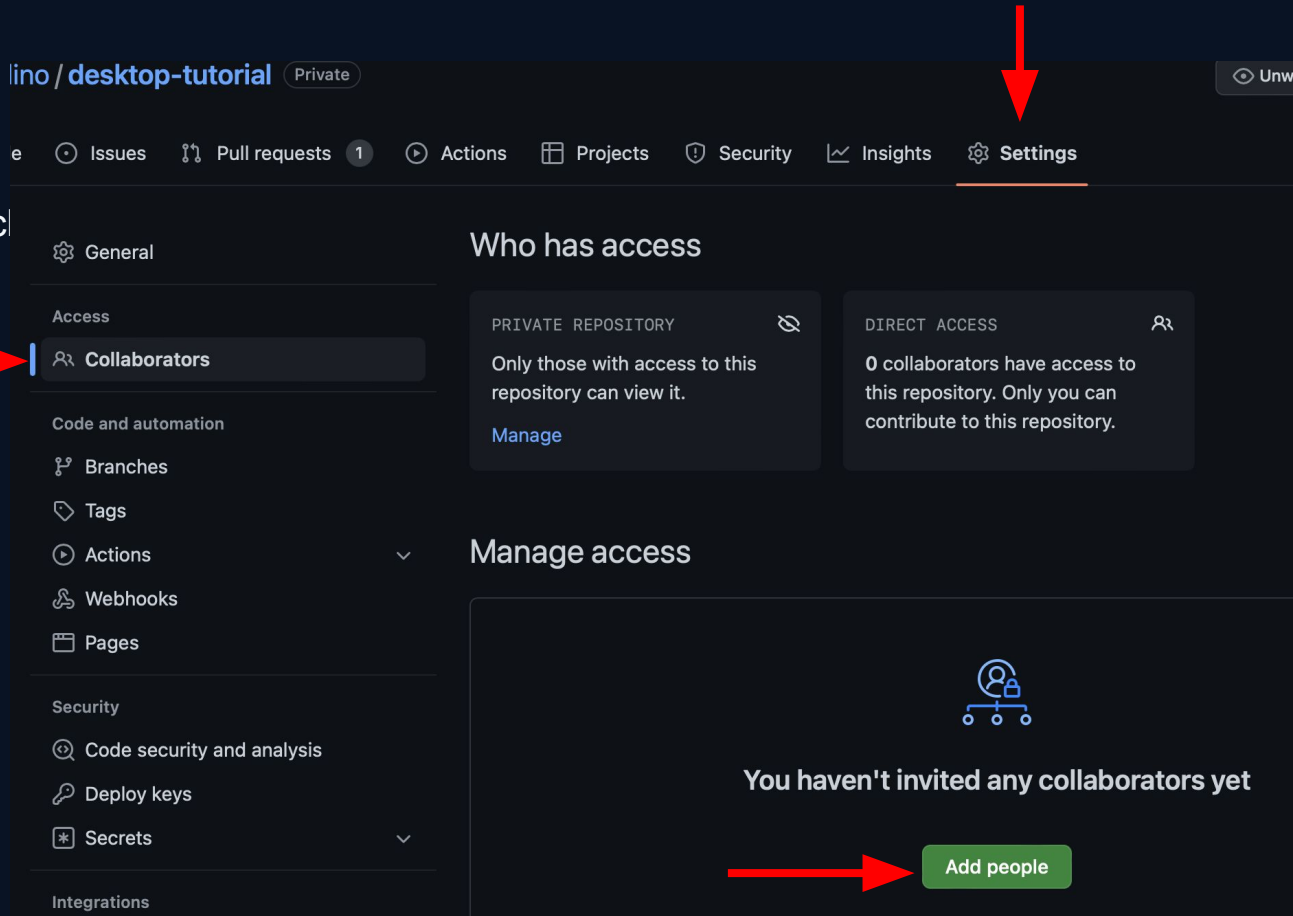


Collaborating

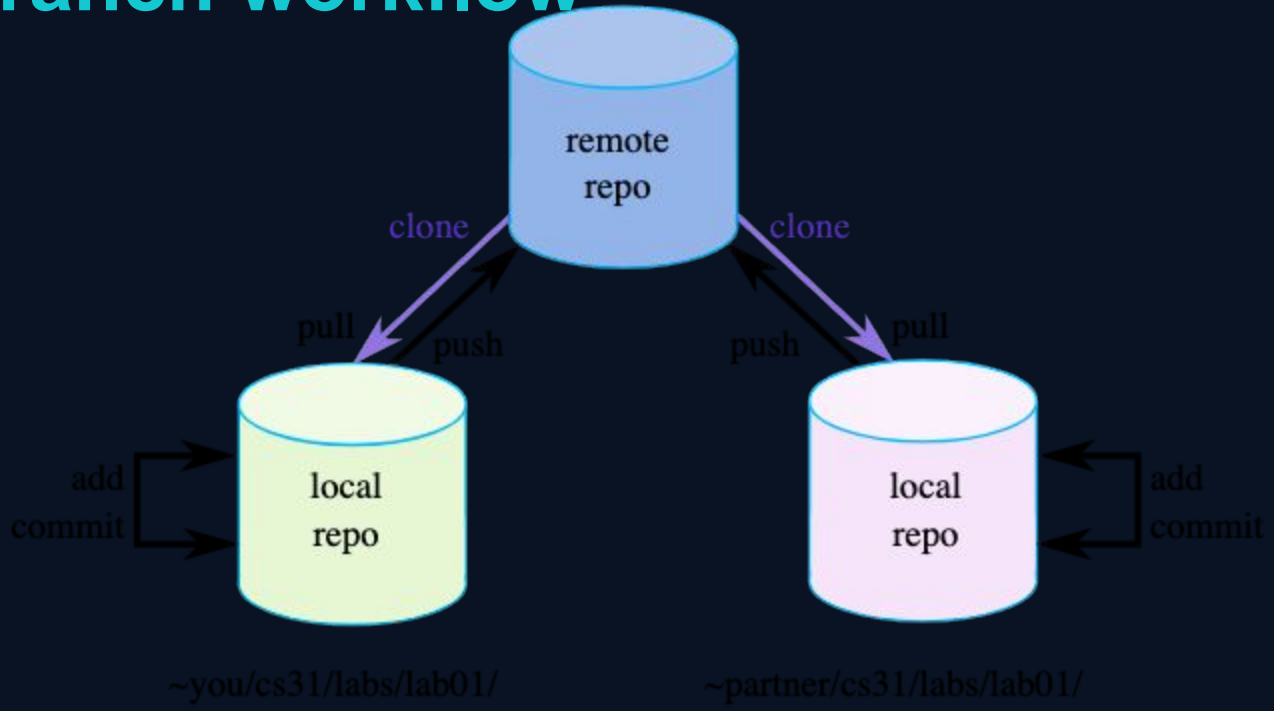
merge

- add changes to other branc

add collaborators



One branch workflow



Going remote

Origin (remote)

- online version of the repository, common for all collaborators

fetch

- load information about online state of the repository

pull

- download changes (mostly new commits)

push

- upload changes (mostly new commits)

Git in the terminal: a safe daily loop

git status	Start here: check branch and changed files.
git switch -c feature/<issue>	Create a short-lived branch.
git diff	Review unstaged changes line by line.
git add -p → git commit	Stage intentionally, then create a local snapshot.
git pull --rebase → git push -u origin HEAD	Integrate first, then publish the branch.

Terminal is explicit and scriptable. Run git status before and after every non-trivial command.

Git reference

Git in VS Code: same repository, visual review

OPEN

Open the repository folder. VS Code detects .git.

BRANCH

Use the status bar to inspect, create or switch branches.

DIFF

Click each changed file and read the side-by-side diff.

STAGE

Stage a whole file or selected lines from Source Control.

COMMIT

Write a clear message and commit locally.

SYNC / PR

Pull before push. Create the PR with the GitHub extension.

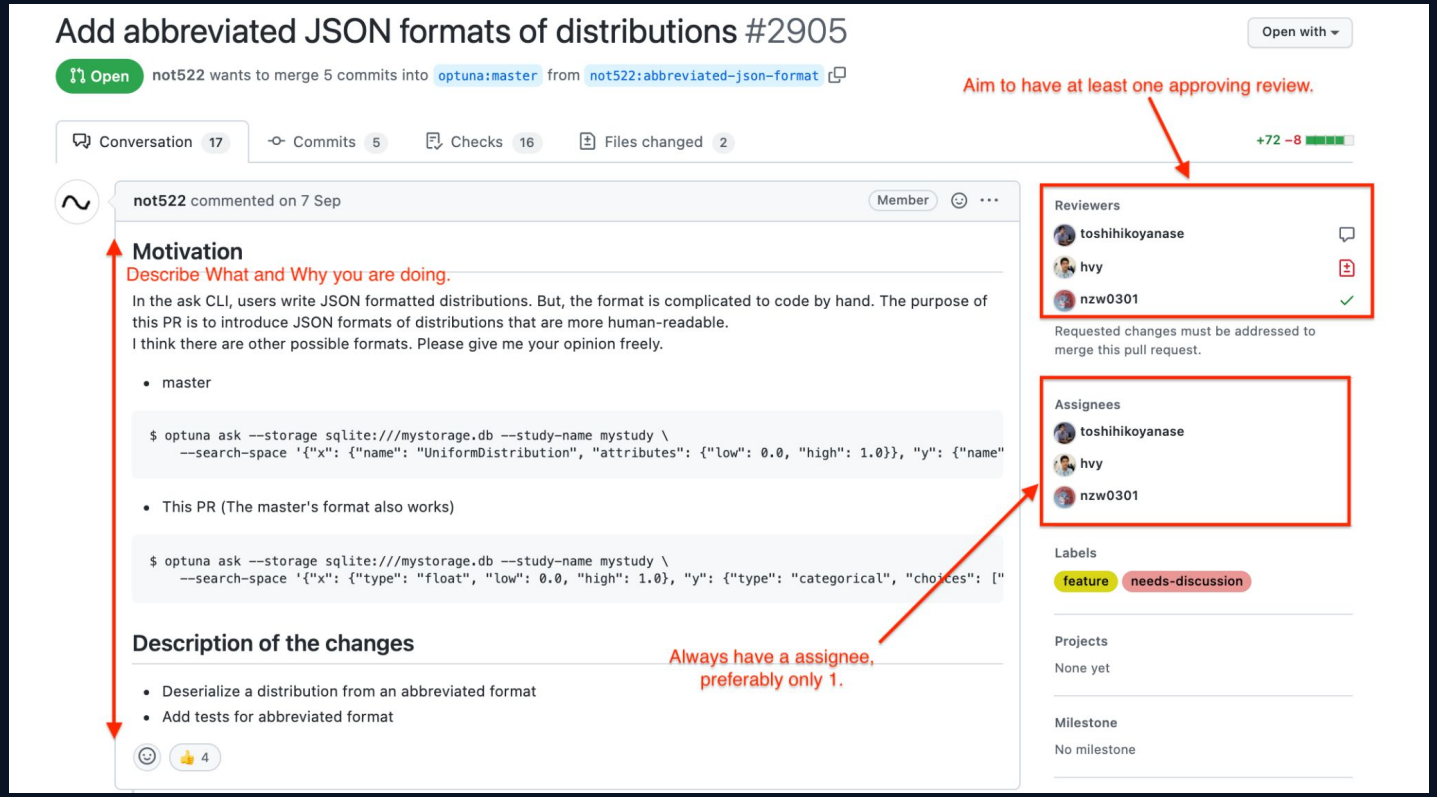
VS Code and the terminal operate on the same .git repository. Choose the view that makes the change easiest to understand.

Merge

- Joining one branch to the other.
- Git tries to merge automatically.

```
22:36 presentation_git_repo on master
➔ git merge address
Auto-merging letter.txt
Merge made by the 'ort' strategy.
 letter.txt | 11 ++++++++++
 1 file changed, 11 insertions(+)
```

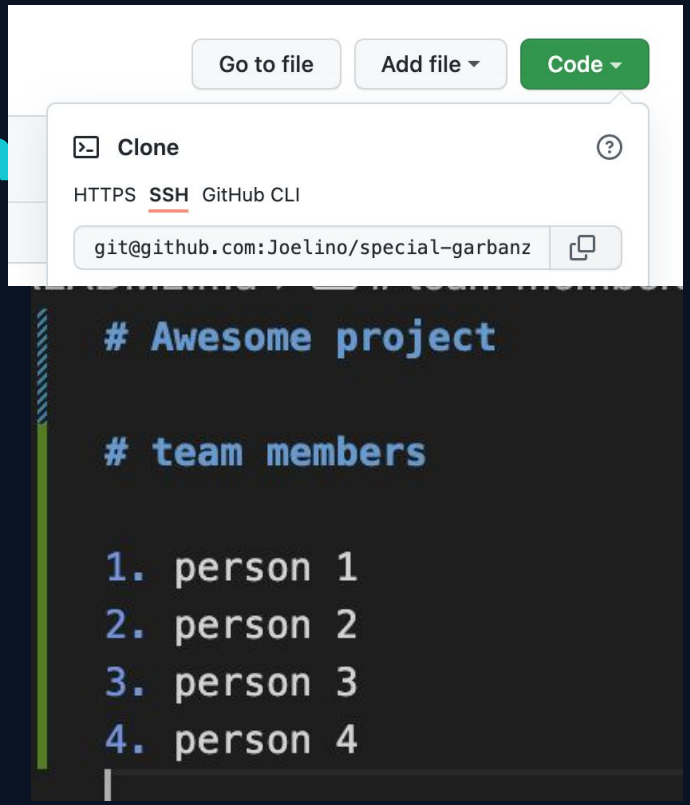
GitHub pull requests - merge with review



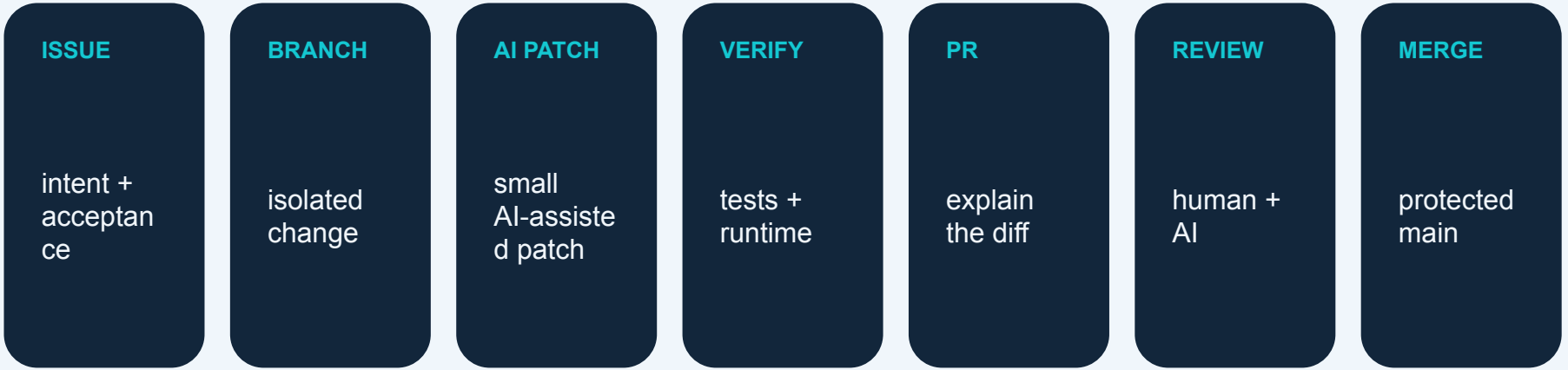
Exercise 1: “Assembling a tea

Set up a repository and add the team.

1. Use private repository rag-fjfi.
2. Create a file with a placeholder team list.
3. Add collaborators.
4. Everyone clones the repository.
5. Each person creates a branch and adds their name.
6. Push branches and open pull requests.
7. Review, approve and merge.



2026 team workflow



Protected main. No direct pushes. Human approval stays mandatory.

GIT LEVEL 2



Three stages of git life

- From idea to immutable snapshot
- Three places where changes to files may live in git
  - Working Directory
  - Stage
  - Repository
    - Local (various branches)
    - Origin (various branches)
    - Other remotes (various branches)

Working directory

"Your playground"

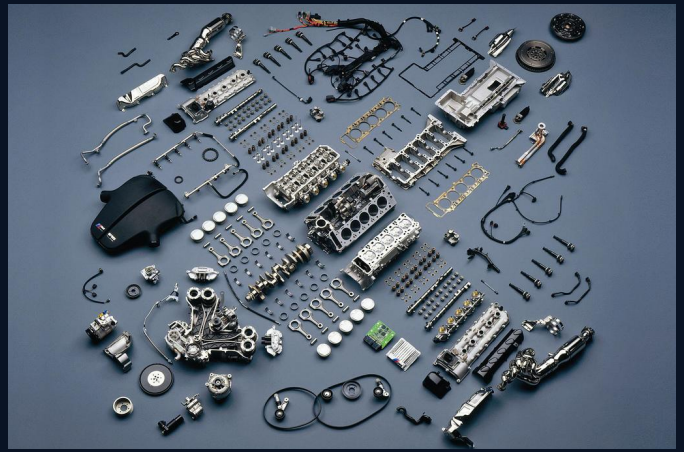
- The current status of the files on your system.
- Modify files as much You want, nothing is stored in git.
- Good for prototyping, experimentation.



Stage (aka Index)

"Putting things together"

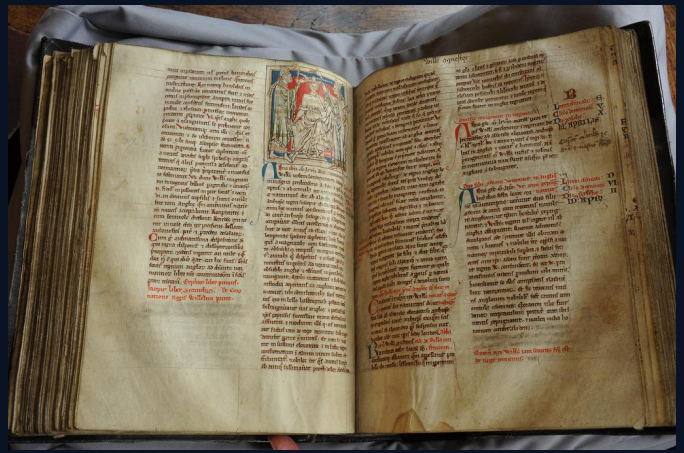
- Created by "adding" (aka "staging") some changes.
- Contains selected changes, from selected files, ready to be committed.
- Allows for grouping related things together in commits



Repository

"Bookkeeping"

- Changes from stage added to repository by committing
- Permanent, immutable (sort of) store of records



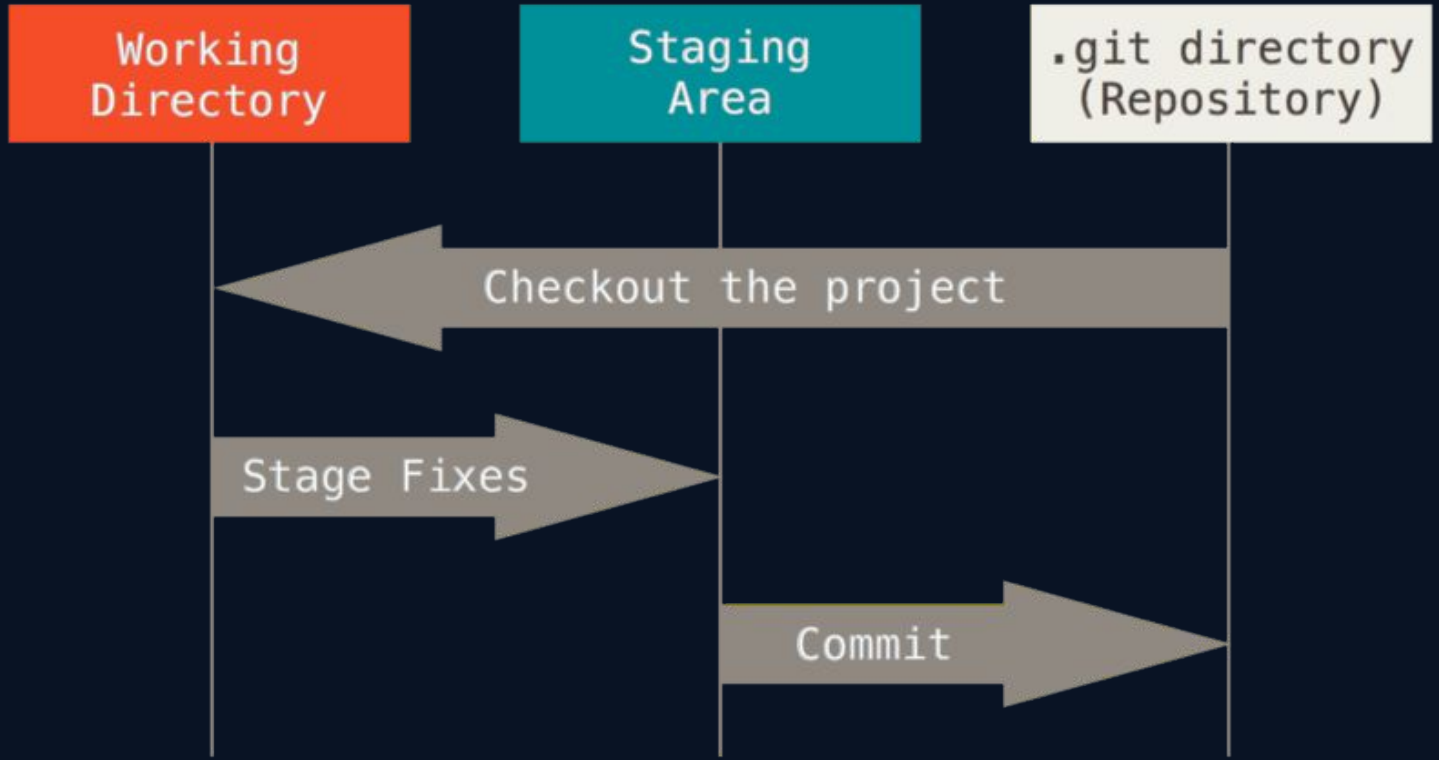
Why bother with staging at all?

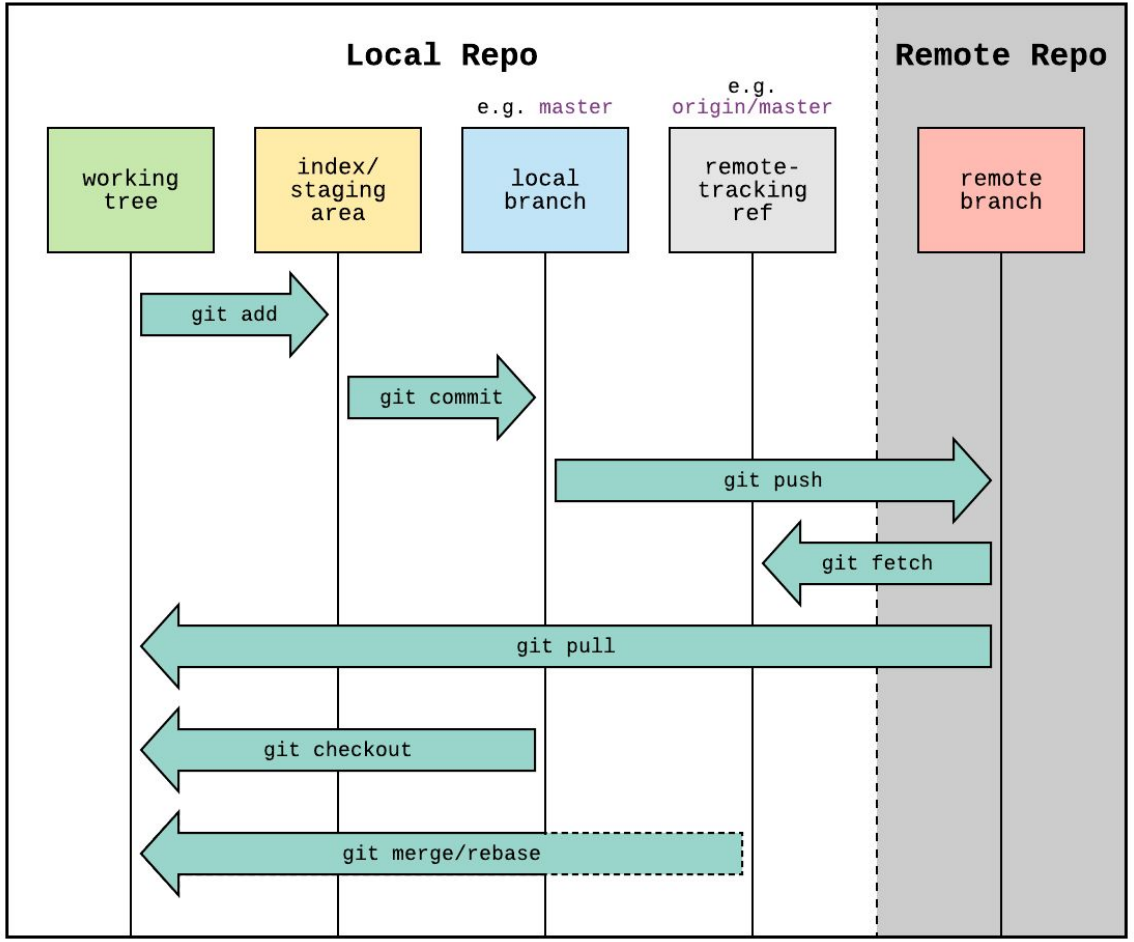
- Lets you select exactly what belongs in the next commit

- Lets you preview the exact changes before committing
  - Groups related changes into one coherent commit

- Working directory can be messy
  - you can easily remove changes you have not staged with “git restore”

Traversing between stages




Local Repo
Remote Repo
e.g. master
e.g. origin/master
working tree
index/ staging area
local branch
remote-tracking ref
remote branch
git add
git commit
git push
git fetch
git pull
git checkout
git merge/rebase

Vibe coding checkpoint: read the diff

INTENT

Does the change solve the issue?

SCOPE

Is the diff small and focused?

TESTS

What evidence proves it works?

SECURITY

Any secrets, unsafe input or permissions?

DEPENDENCIES

Why is every new package needed?

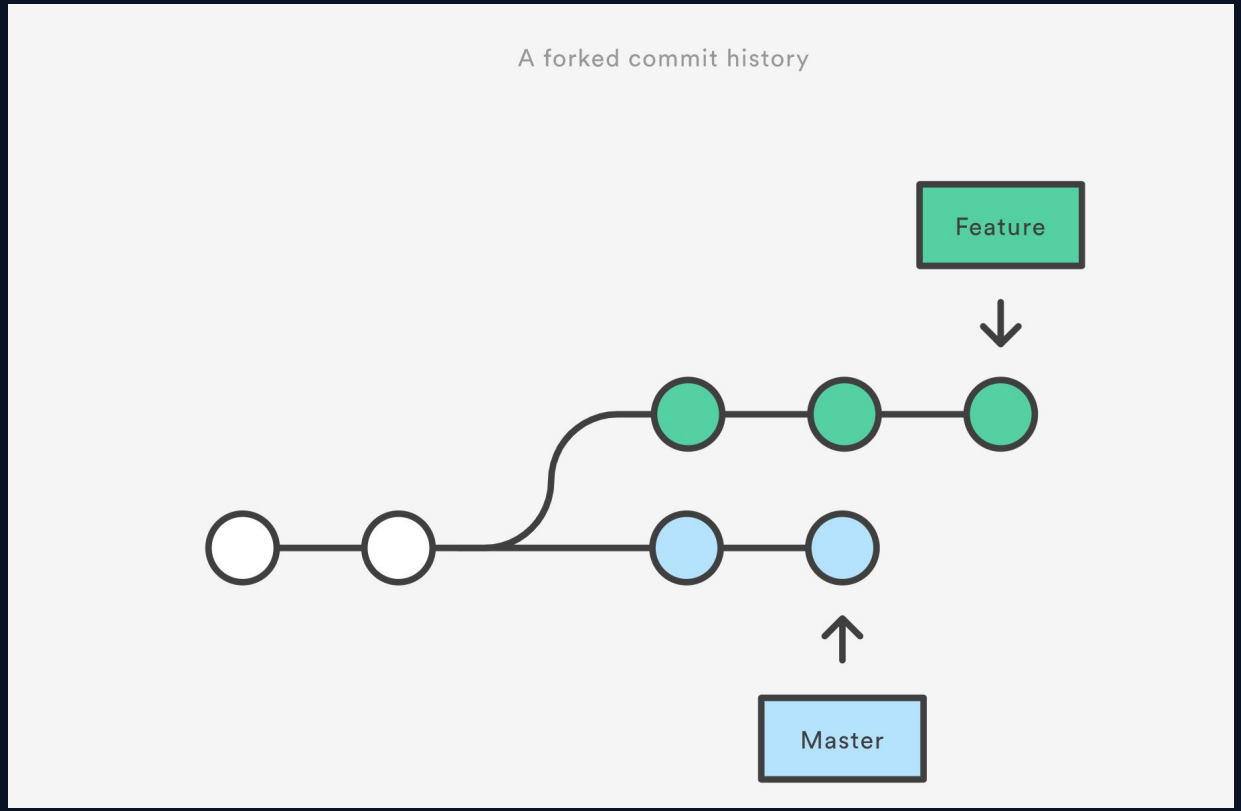
REVERSIBILITY

Can we safely revert the change?

If you cannot explain the diff, you are not ready to merge it.

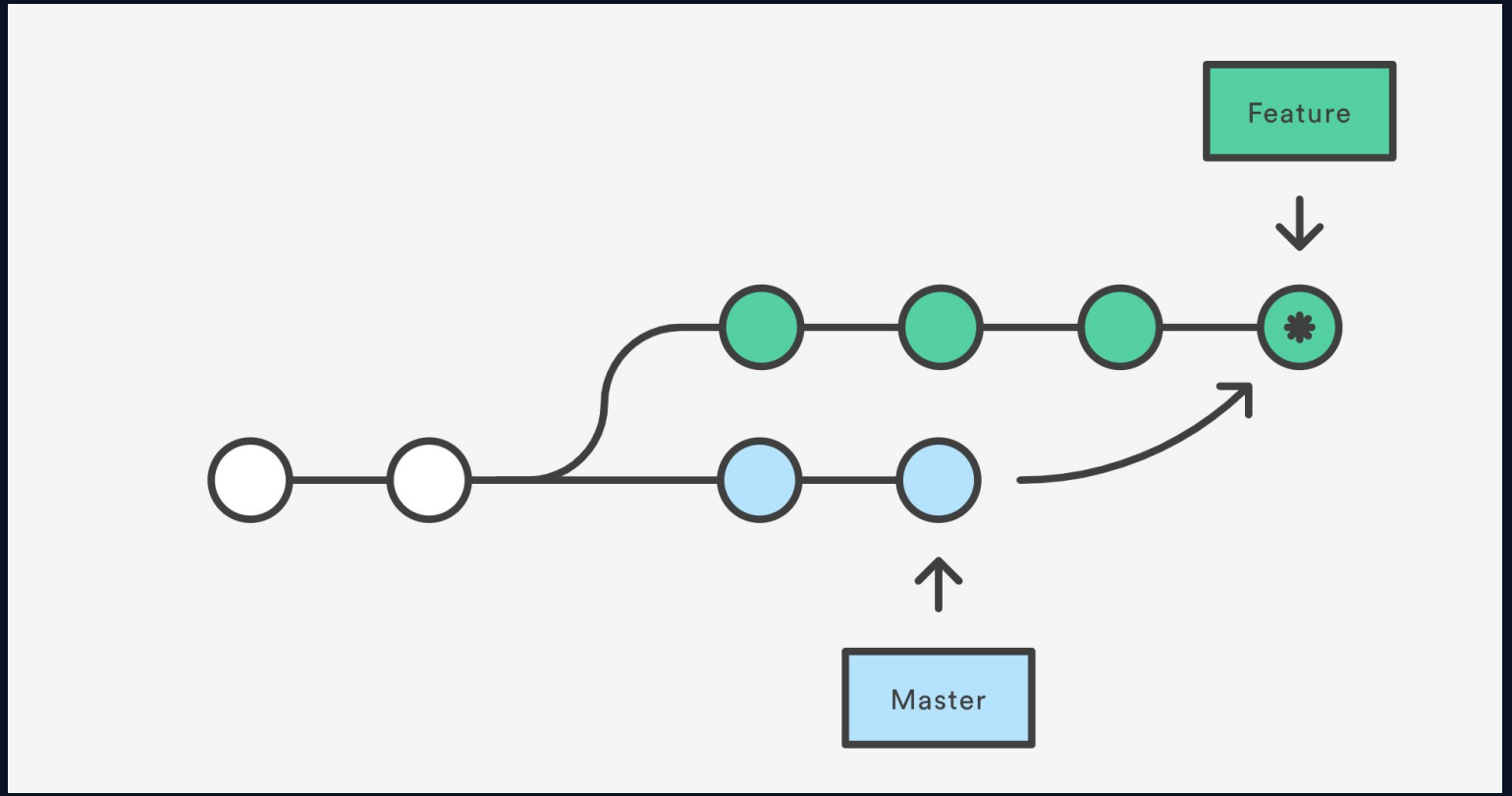
Merging changes

rebase vs merge



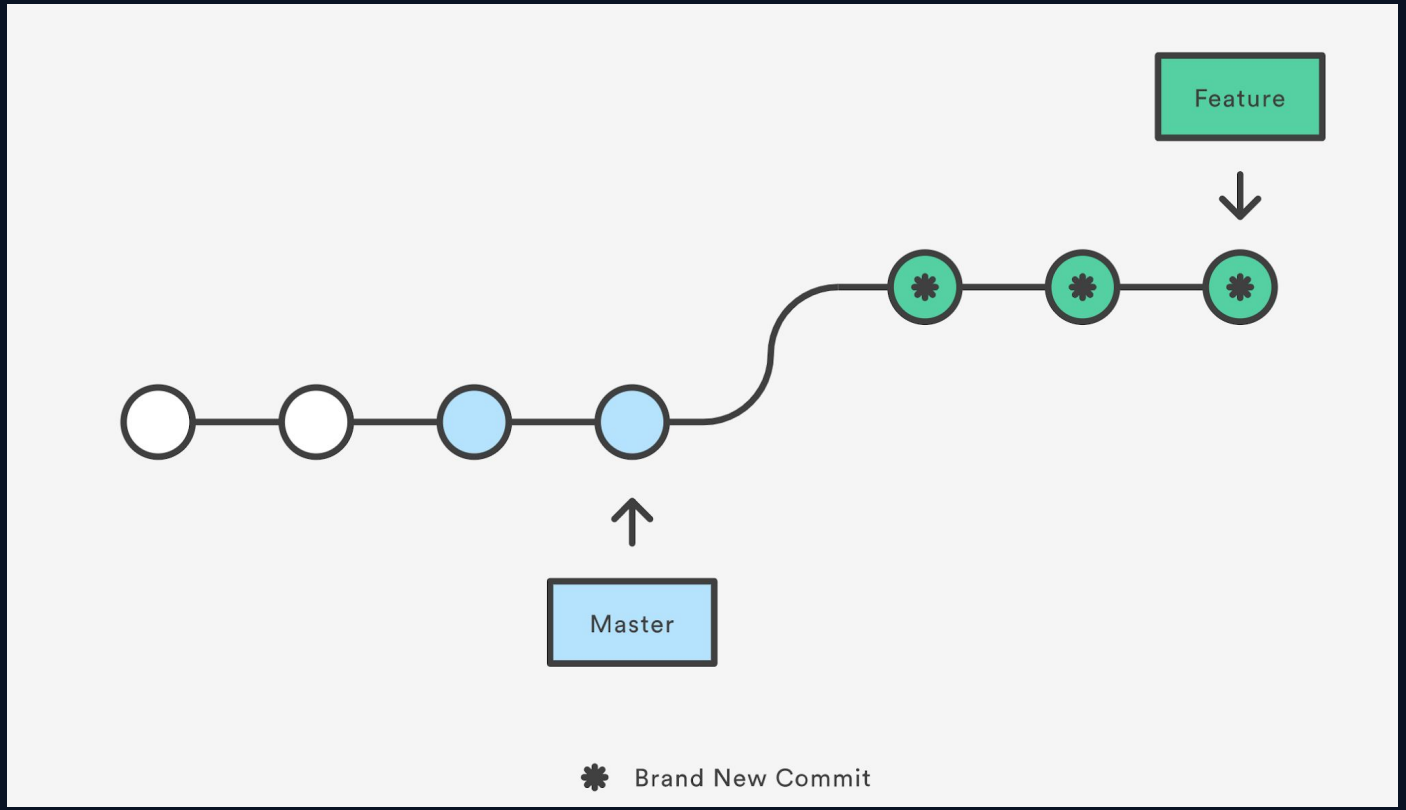
source

rebase vs merge



source

rebase vs merge



source

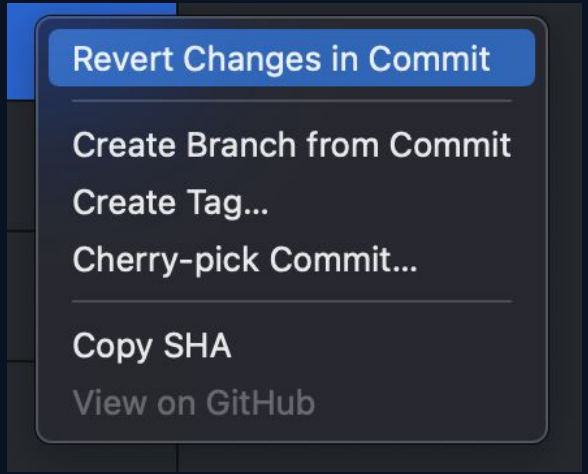
Handling conflicts

```
1   RATE = 0.05
2
3   account = 100
4   additive = False
5
6
7   for year in range(10):
    Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
8   <<<<<<< HEAD (Current Change)
9       account *= 1 + RATE
10  =======
11      if additive:
12          account += 0.5
13      else:
14          account *= 1.05
15  >>>>>>> feature (Incoming Change)
16
```

Exercise 3: “Conflicting poet”

Try to create a conflict and resolve it.

1. Go to repo from exercise 1.
2. Create a disposable branch before experimenting.
3. Revert the removal commit
4. Inspect the conflict markers and the diff.
5. Manually handle the conflict and commit the changes.



When AI suggests a Git command

1 READ

Understand every flag.

2 CHECK

Run git status and git diff.

3 PREDICT

Name what will change.

4 PREFER

Choose the reversible option.

5 INSPECT

Verify the result immediately.

Never run reset --hard or clean -fd without understanding the exact targets.

Git Best practices



Git Best practices

Some opinions on how git should be used:

1. Use short-lived branches and merge frequently
2. Commit small, coherent changes
3. Check git status and git diff before acting
4. Push branches and protect main
5. Never rewrite shared history
6. Explain what changed, why and how it was verified

more at https://sethrobertson.GitHub.io/GitBestPractices/

GitHub Best Practices

- Develop on short-lived branches and merge through pull requests.
- Require human review and automated checks.
  - But do that like a good person! https://mtlynch.io/human-code-reviews-1/ and https://mtlynch.io/human-code-reviews-2/ .
- Maintain README.md
  - How to install.
  - How to run.
  - How to contribute.
- Maintain .gitignore
  - Never commit virtual environments, secrets, local configuration or private data.

Give coding agents repository context

README.md

Purpose, setup and commands.

AGENTS.md

Architecture, constraints and local rules.

COPILOT INSTRUCTIONS

Review criteria and coding conventions.

CONTRIBUTING + TESTS

The workflow and executable definition of done.

Good context reduces random edits. It does not remove the need for review.

GitHub Docs: customize Copilot

Safe recovery toolbox

1. DISCARD A LOCAL FILE: git restore <file>
2. UPDATE LAST COMMIT: git commit --amend
3. REBASE INTERACTIVELY: git rebase -i
4. FIND LOST COMMIT / STATE: git reflog

best resource (bookmark this) https://ohshitgit.com/

Prefer git revert for changes already shared with the team.

Virtual envs and package management

•••

Reproducible development environments

Packages

- Huge part of useful functionality of Python delivered in packages.
- Packages depend on other packages.
- Packages get updated all the time.
- Often some updates are non-backward-compatible.
- We need to manage dependencies across the project.
    - The same we manage code, we need to manage packages.

AI can add dependencies faster than you can review them

NEED

Can the standard library or existing stack do it?

SOURCE

Is the package maintained and trusted?

VERSION

Is the constraint intentional?

LOCK DIFF

Review transitive changes in uv.lock.

SECURITY

Check advisories, licenses and secrets.

Use uv add. Review pyproject.toml and uv.lock together.

Before uv: the manual Python workflow

python -m venv .venv	Create an isolated environment.
source .venv/bin/activate	Activate it manually. Windows used a different command.
python -m pip install <package>	Install into the currently active environment.
python -m pip freeze > requirements.txt	Capture the current package versions manually.
deactivate	Leave the environment and activate it again next time.

Still useful to understand, but easy to drift. In this course uv replaces these manual steps.

Python venv docs

Package manager pip

- Install package: pip install
  - >>> pip install pandas
- specific version
  - >>> pip install pandas==0.25.1
- List version of package: pip show
  - >>> pip show pandas
  - Name: pandas
  - Version: 0.25.1
  - Summary: Powerful data structures for data analysis, time series, and statistics

Package manager pip

- List installed packages: pip list
  - >>> pip list
  - Package Version
  - ---------- -------
  - numpy 1.17.2
  - pandas 0.25.1
  - pip 19.0.3
- Upgrade package: pip install --upgrade
  - >>> pip install pandas --upgrade
- Save installed packages to file: pip freeze
  - >>> pip freeze > requirements.txt



Package manager pip

```
>>> cat requirements.txt
numpy==1.17.2
pandas==0.25.1
python-dateutil==2.8.0
pytz==2019.2
six==1.12.0
```

- Install packages from file pip install -r
  - >>> pip install -r requirements.txt



Virtual environment

is a sandbox that contains

- a Python installation for a particular version of Python,
- plus a number of additional packages.

Why?

- To gain reproducibility
- between your different machines
- between you and your colleagues
- between your dev machines and machines in production environments

Why cont.

It's very easy to get bugs because of mismatching package versions. Exceptions or subtler issu

```
import pandas as pd
import numpy as np

print("pandas version: {}".format(pd.__version__))

monthly_rev = pd.Series([np.nan, np.nan, 1, 5, 10, 20])
```

```
rev_to_date = monthly_rev.expanding(min_periods=0).sum()
monthly_pct_increase = rev_to_date.diff(1) / rev_to_date
avg_pct_increase = monthly_pct_increase.mean()

pretty_number = int(round(avg_pct_increase * 100))
print("Average Revenue Growth Rate is {} %.".format(pretty_number))
```

Why cont.

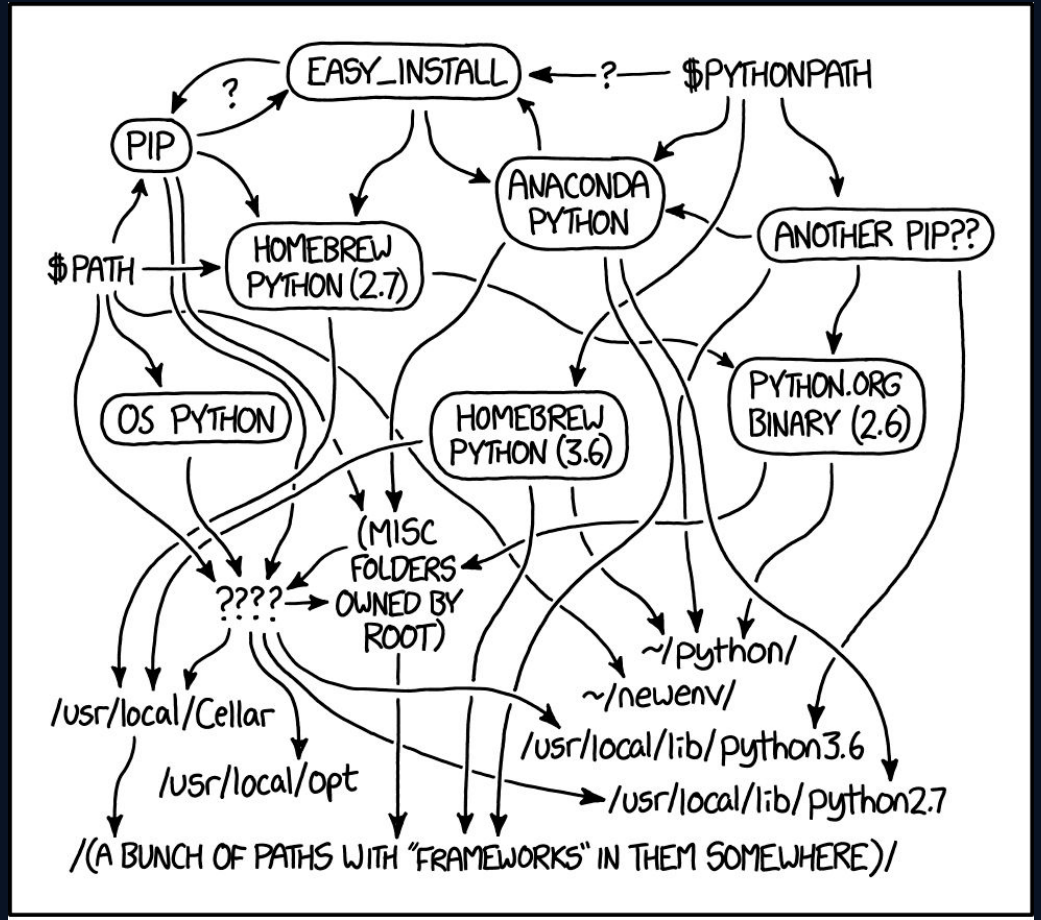
```
pandas version: 0.18.1
Average Revenue Growth Rate is 67 %.
```

VS

```
pandas version: 0.25.1
Average Revenue Growth Rate is 75 %.
```

Why cont.

- What if you are working on multiple projects with different requirements at the same time?
- What if you want to run your project in two kinds of environments? (e.g. development and production)



MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.

Foundation: Python virtual environments

- venv is Python's built-in isolation mechanism.
  - others: conda, virtualenv(wrapper)...
  - there are tools to also handle multiple version of python itself (pyenv)

- Course default in 2026: let uv create and manage .venv automatically.

How venv works underneath

Create env

> python3 -m venv <DIR>

- Creates an isolated environment directory.
-

Activate

> source <DIR>/bin/activate

Deactivate

> deactivate

In this course, uv manages .venv for you.

Windows setup

Use native PowerShell or WSL. Pick one setup and document it for the team.

How To Set Up A Python Virtual Environment On Windows 10

Managing Python versions

- Course default: uv python install 3.14 and declare requires-python in pyproject.toml.
- Advanced alternative: pyenv or a system package manager.
  - guide: https://realpython.com/intro-to-pyenv/#installing-pyenv
  - Windows? Use WSL or https://GitHub.com/pyenv-win/pyenv-win (not tested by me)

Exercise 4: “Works on my machine”

Add script to your repo that allows you to check that you use the same Python and library version.

One person from the team:

1. Create the project environment.
    a. Use Python 3.14 unless the project requires a different supported version.
    b. > uv sync
    c. Resources:
        i. Python Virtual Environments: A Primer - Real Python
        ii. How to Upgrade Python to 3.9 { Windows, macOS and Linux}
2. Run commands through uv
    a. > uv run python --version
3. Add a dependency
    a. > uv add flask

Exercise 4: “Works on my machine” cont.

One person from the team cont.

4. Download the test files and put them into repo.
5. Commit pyproject.toml and uv.lock.
    a. > uv lock --check
6. Run the checks through uv.
    a. > uv run python version_check.py
7. Commit the changes.
8. Push the changes.

Exercise 4: “Works on my machine” cont.

Rest of the team

1. Clone the repository.
2. Create the exact environment from the lockfile.
   a. > uv sync --locked
3. Run the checks through uv.
   a. If the package check is not passing … fix it, you did something wrong.
   b. If the Python check is not passing … you have homework to upgrade. See How to Upgrade Python to 3.9 { Windows, macOS and Linux} or Google.

Cmd output example
https://gist.GitHub.com/Joelino/d2e3176a4fdb9debad1fdd24457f9fed

Optional Exercises

- https://learngitbranching.js.org/ (from exercise 1.4?)
- https://gitexercises.fracz.com/ (from exercise 3-4?)
- self-help group

We didn’t have time for...

- Which editor to pick (beginning of file)
- Python best practices

Modern Python project setup (2026)

- Goal: fast installs, reproducible environments, easy team onboarding
- Course default: uv manages Python, dependencies and .venv.
- This deck: install, sync, run, add, update and review

Why move beyond pure pip/venv?

- pip installs but doesn’t lock exact versions → “works on my machine”
- requirements*.txt tend to drift, manual pinning is tedious
- Multiple tools needed (pip + venv + pip-tools) complicate onboarding
- Need for deterministic installs and single-command workflows

2026 default: uv-managed Python project

uv python install 3.14	Choose the interpreter
uv sync --locked	Reproduce the environment
uv add <package>	Manage a dependency
uv run pytest	Run inside the environment
uv lock --check	Verify metadata and lock agree

Commit pyproject.toml and uv.lock. CI should use the locked environment.

Astral uv Docs

What is uv?

- One tool for Python versions, dependencies, lockfiles and virtual environments
- Written in Rust → very fast resolves/installs
- Core ideas:
  - pyproject.toml declares what you want
  - uv.lock captures exact, reproducible resolution
  - uv sync creates .venv and installs the locked environment
  - uv run executes inside it, without manual activation
  - uv lock --upgrade-package <name>

Join an existing project with uv

git clone <url>	Get the repository, pyproject.toml and uv.lock.
cd <repo> → uv sync --locked	Create .venv and install the locked dependencies.
uv run pytest	Run tests inside the managed environment.
uv run python -m <module>	Run the application without manual activation.
uv lock --check	Verify that project metadata and lockfile agree.

Do not edit uv.lock by hand. Commit pyproject.toml and uv.lock together.

Astral uv Docs

Change dependencies safely with uv

ADD

uv add <package>

DEV TOOL

uv add --dev pytest ruff

REMOVE

uv remove <package>

UPDATE ONE

uv lock --upgrade-package <package>

VERIFY

uv lock --check → uv run pytest

REVIEW

git diff -- pyproject.toml uv.lock

Small dependency diffs are easier to review. Check need, source, version constraints and the lockfile.

Comparison

Tool	Role	Strengths	Limits	Use when…
pip	installer	ubiquitous	no lock by default	small scripts, legacy
venv	isolation	stdlib	no dep mgmt	quick, manual envs
pipenv	deps + env	simple start	less momentum	small apps
conda	deps + env (multi-lang)	strong for data/GPU	heavier, outside PyPI flow	scientific stacks
uv	project + deps + env + Python	fast, **lock/sync**, one tool	newer	modern app/dev workflow

```
# Verify the team tool
uv --version

# New project
uv init hello-world
cd hello-world
uv run python main.py     # runs inside managed env
```

What pyproject.toml looks like

[project]

name = "demo"

version = "0.1.0"

requires-python = ">=3.12"

dependencies = ["fastapi", "uvicorn"]

[dependency-groups]

dev = ["pytest", "ruff"]

Keep pyproject.toml + uv.lock in git

```
[project]
name = "pdecay-masters"
version = "0.1.0"
description = "DUNE proton-decay classification training utilities and scripts."
readme = "README.md"
requires-python = ">=3.12"
authors = [{ name = "Jiri Franc" }]
dependencies = [
    "numpy>=2.1",
    "scipy>=1.12",
    "pandas>=2.1",
    "torch>=2.4",
]
```

Export to requirements.txt

uv export --format requirements.txt --frozen --no-hashes --output-file requirements-simple.txt

Only for legacy tools that do not understand pyproject.toml

Commit pyproject.toml and uv.lock. Do not hand-edit the lockfile.

Exercise: AI-assisted, human-owned change

1. Write an issue with acceptance criteria.
2. Ask AI for a plan and identify risks.
3. Create a branch and generate one small patch.
4. Read every line of the diff.
5. Run tests and reproduce the behavior.
6. Open a PR and explain your decisions.

AI may propose. You own the commit.